Name: D.VAISHNAVI(192524143)

ASSIGNMENT 1

CO1: *Java fundamentals and object-oriented programming – data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation*

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks - 50)

Submission Date: _____22/10/2026_____

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: **https://github.com/vaishu1104/CSA0909-192524143-Assignment1**

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

TASK 1 — Understand the Problem

Aim: To analyse the Campus Smart-Parking System and identify the objects, classes, inputs, outputs, responsibilities, business rules, and assumptions required to design the system using Java.

1. Inputs and Outputs Inputs:

- Vehicle registration number.
- Vehicle type (Two-wheeler, Car, or Electric Vehicle).
- Floor number and parking slot number.
- Vehicle entry time and exit time.
- User type (Student, Staff, or Visitor).
- Monthly pass validity.
- Number of floors and slots available on each floor.

Outputs:

- Allocated parking floor and slot number.
- Unique parking ticket number.
- Vehicle entry time and exit time.
- Total parking duration.
- Parking fee based on vehicle type and duration.
- Charging fee for electric vehicles, where applicable.
- Slot availability status.
- Error message when no suitable slot is available or invalid input is provided.

2. Candidate Classes

A class is a blueprint used to create objects in Java.

Candidate class	**Why it is needed**
Vehicle	Represents the common details of a vehicle.
TwoWheeler	Represents a two-wheeler and its parking rate.
Car	Represents a car and its parking rate.
ElectricVehicle	Represents an electric vehicle with charging facilities.
ParkingSlot	Represents an individual parking slot and its availability.
ParkingFloor	Represents a floor containing multiple parking slots.

ParkingTicket	Stores the ticket number, vehicle details, and entry and exit times.
User	Represents the person using the parking facility.
Student, Staff, Visitor	Represent different user categories and their parking privileges.
ParkingFeeCalculator	Calculates the parking fee based on the applicable rules.

These are candidate classes. The final design may combine or modify some classes depending on the implementation.

3. Responsibilities of Each Class

class	**Responsibilities**
Vehicle	Stores registration number and defines common vehicle behaviour.
TwoWheeler	Defines the parking rate for two-wheelers.
Car	Defines the parking rate for cars.
ElectricVehicle	Supports EV charging and charging fee calculation.
ParkingSlot	Stores slot number, slot type, and occupancy status.
ParkingFloor	Maintains the slots on a particular floor.
ParkingTicket	Finds a free slot, allocates it, and releases it when the vehicle leaves.
User	Stores user details and pass information.
Student, Staff, Visitor	Identify the user category and applicable parking privileges.
ParkingFeeCalculator	Calculates the final parking fee according to the rules.

Important methods

- allocateSlot() – Finds and assigns a free parking slot.
- releaseSlot() – Makes a slot available after the vehicle leaves.
- generateTicket() – Creates a parking ticket with a unique number.
- calculateFee() – Calculates the parking charge.
- isPassValid() – Checks whether the user's monthly pass is valid.
- findVehicle() – Searches for a vehicle using its registration number.

4. Business Rules

Business rules are conditions that the system must follow during every valid operation.

- A parking slot can hold only one vehicle at a time.
- A vehicle must be assigned a suitable free slot.
- Every parking ticket must have a unique ticket number.
- A ticket number must never be reused.
- A vehicle cannot be allocated to an occupied slot.
- When a vehicle leaves, its slot must be released.
- The parking fee depends on the vehicle type and parking duration.
- Students and staff with a valid monthly pass can park free of charge.
- Visitors must pay the full parking fee.
- Electric vehicle charging fees must be calculated separately.
- Invalid floor numbers and slot numbers must be rejected.
- If no suitable slot is available, the system must report that parking is full.

5. Assumptions

- Every vehicle has a unique registration number.
- The parking facility has a fixed number of floors and slots during a parking session.
- Each slot can accommodate only one suitable vehicle.
- Entry and exit times are recorded using the system clock.
- Parking charges are calculated using the rates defined by the university.
- Monthly passes are checked for validity before applying free parking.
- Electric vehicle charging is available only in designated charging slots.
- The system processes one parking operation at a time for this initial design.
- The number of slots can differ from one floor to another.
- Ticket numbers are generated using a counter that guarantees uniqueness during the system's operation.

Comparison

Feature	Vehicle objects	Parallel arrays
Data storage	Each vehicle is an object.	Data is stored in separate arrays.
Finding a vehicle	Search using the object's registration number.	Search the regNo[] array.
Adding a new attribute	Add a field to the Vehicle class.	Add another array and maintain matching indexes.
Maintainability	Easier to organise and extend.	More difficult as attributes increase.
Data safety	Encapsulation can protect the fields.	Data consistency must be maintained across arrays.

Time complexity of searching

Suppose there are \(n\) parked vehicles.

- Vehicle objects: Linear search takes \(O(n)\) time in the worst case.
- Parallel arrays: Searching regNo[] also takes \(O(n)\) time in the worst case.

Both approaches have the same worst-case search complexity when using linear search.

Space complexity

- Vehicle objects: \(O(n)\) space for \(n\) vehicles, including their object data and references.
- Parallel arrays: \(O(n)\) space for \(n\) vehicles, with separate storage for each attribute.

The exact memory usage depends on the Java implementation and the number and types of attributes.

Conclusion

The object-oriented approach is better for the Campus Smart-Parking System because it groups related data and behaviour in classes. It supports encapsulation, inheritance, and polymorphism, making the system easier to maintain and extend when new vehicle types or parking rules are introduced.

TASK 2 – Choose the Data Representation

Aim: To select suitable Java data types and arrays to represent the floors, parking slots, vehicle details, entry time, parking duration, and parking fee. 1. Data Types Used

Data	Java data type	Reason
Floor number	int	Stores the floor index.
Slot number	int	Stores the slot index.
Vehicle registration number	String	Stores letters and numbers.
Vehicle type	String or enum	Identifies the vehicle category.
Entry time	LocalDateTime	Stores the date and time of entry.
Exit time	LocalDateTime	Stores the date and time of exit.
Duration	long	Stores parking duration in minutes.
Parking fee	double	Stores the calculated fee, including decimal values.
Ticket number	long	Stores a unique numeric ticket number.
Slot availability	boolean	Indicates whether a slot is free or occupied.

2. Representation of Floors and Slots

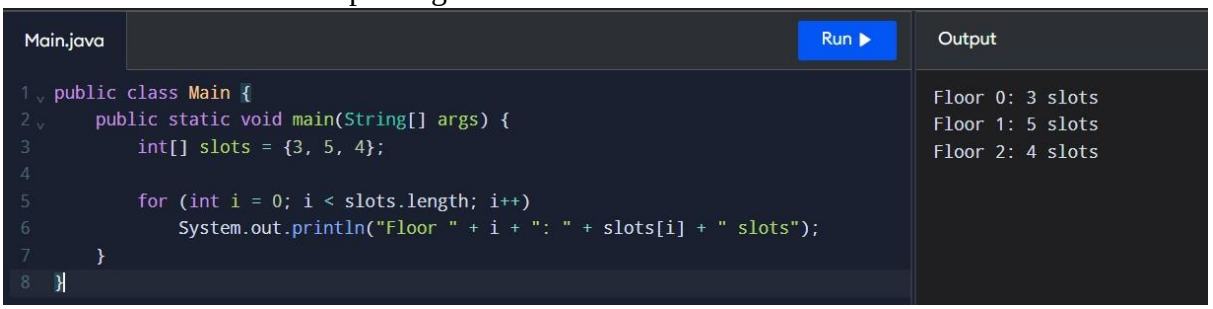
I would use a jagged two-dimensional array because each floor may contain a different number of slots.

```
String[][] slots = {
new String[3],
new String[5],
new String[4]
};
```

Here:

- slots[0] contains 3 slots.
- slots[1] contains 5 slots.
- slots[2] contains 4 slots.
- A null value indicates that a slot is free.
- A non-null value stores the registration number of the parked vehicle.

This representation is suitable because different floors can have different numbers of parking slots.



3. Type Conversion and Casting

Type conversion is required when calculating the parking duration and fee.

For example, if a vehicle is parked for 90 minutes, the duration is converted into billable hours. If every started hour is charged in full, 90 minutes must be charged as 2 hours.

int minutes = 90; int hours =
(minutes + 59) / 60;
System.out.println(hours);

Output:

2

The formula (minutes + 59) / 60 performs ceiling division for non-negative minutes using integer arithmetic.

4. Risk of Integer Division

Consider this incorrect calculation:

int minutes = 90; int hours =

minutes / 60;

System.out.println(hours);

Output:

1

This is incorrect because integer division discards the fractional part. The correct billable duration is 2 hours.

We can avoid this error by using:

int hours = (minutes + 59) / 60;

For a fee of ₹30 per hour, the correct fee is:

\[2 \times 30 = ₹60 \]

5. Reading Input

The Scanner class can be used to read the vehicle registration number and other details from the user.

```
import java.util.Scanner; class

ParkingInput {     public static void

main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter vehicle number: ");

        String regNo = sc.nextLine();

        System.out.print("Enter parking duration in minutes: ");

int minutes = sc.nextInt();

        System.out.println("Vehicle: " + regNo);

        System.out.println("Duration: " + minutes + " minutes");

        sc.close();

    }

}
```

Sample Input:

TN01AB1111

90

Sample Output:

Vehicle: TN01AB1111

Duration: 90 minutes

6. Alternative Concept and Complexity Analysis

Instead of a jagged array, the system could use a one-dimensional array or an ArrayList<String[]>.

Comparison

Representation	Advantages	Limitations
Jagged array	Each floor can have a different number of slots.	Requires nested loops to search all slots.
One-dimensional array	Simple storage and traversal when all slots are mapped to one index.	Needs index calculations and extra mapping for unequal floor sizes.
ArrayList<String[]>	Flexible collection of floor arrays.	Slightly more collection overhead.

For a one-dimensional array with equal slots per floor, the index can be calculated as:

index = floor * slotsPerFloor + slot

When floors have different numbers of slots, the simple formula does not work directly. Prefix offsets or a mapping table are required.

Time and Space Complexity

Let F be the number of floors and S the maximum number of slots per floor.

- Searching for a slot: Worst-case time is $O(FS)$ for a jagged array.
- One-dimensional array: Searching all slots takes $O(N)$, where N is the total number of slots.
- Space complexity: Both representations require $O(N)$ space for N total slots.
- ArrayList: Also requires $O(N)$ space, with additional collection-management overhead.

Result

Thus, suitable Java data types and a jagged two-dimensional array are selected to represent the Campus Smart-Parking System. Correct integer arithmetic prevents billing errors, and the design supports floors with different numbers of parking slots.

TASK 3 – Design the Control Logic

Aim

To design the control logic for allocating a free parking slot and calculating the parking fee using Java.

1. Slot Allocation – Pseudocode

ALGORITHM AllocateSlot(vehicleNo)

1. FOR each floor f
2. FOR each slot s in floor f
3. IF slot is free THEN
4. Store vehicleNo in the slot
5. RETURN {f, s}
6. END IF
7. END FOR
8. END FOR
9. RETURN null

END ALGORITHM

2. Parking Fee Calculation – Pseudocode

The assignment does not specify exact parking rates. The following pseudocode uses an hourly rate and a daily maximum as examples.

ALGORITHM CalculateFee(minutes, ratePerHour)

1. IF minutes <= 0 THEN
2. RETURN 0
3. END IF
4. SET hours = (minutes + 59) / 60
5. SET fee = hours * ratePerHour
6. IF fee > dailyMaximum THEN
7. SET fee = dailyMaximum
8. END IF
9. RETURN fee

END ALGORITHM

3. Complete the Code – ParkingFacility.java public class ParkingFacility {

```
   // slots[floor][slot] holds a vehicle number, or null if free
private final String[][] slots;     public ParkingFacility(int[]
slotsPerFloor) {         slots = new
String[slotsPerFloor.length][];         for (int f = 0; f <
slotsPerFloor.length; f++) {             slots[f] = new
String[slotsPerFloor[f]];
       }
   }
   /** Returns {floor, slot} of the first free slot, or null if full. */
public int[] allocate(String vehicleNo) {         for (int f = 0; f <
slots.length; f++) {             for (int s = 0; s < slots[f].length;
s++) {                if (slots[f][s] == null) {                    slots[f][s]
= vehicleNo;                    return new int[]{f, s};
              }
          }        }
return null;
   }
   /** Frees a slot. Returns false if invalid or already free. */
public boolean release(int floor, int slot) {         if (floor < 0
|| floor >= slots.length)             return false;         if (slot < 0
|| slot >= slots[floor].length)             return false;

       if (slots[floor][slot] == null)
return false;         slots[floor][slot]
= null;         return true;
   }
   public static void main(String[] args) {
```

```
        // floor 0 has 2 slots, floor 1 has 1 slot
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```



4. Expected Output

[0, 0]

[0, 1] [1,

0] null

true

false

false

[0, 1]

5. Explanation of Control Statements

- for loop: Searches each floor and each slot.
- if statement: Checks whether a slot is free or a position is valid.

- return: Returns the allocated position or stops the method when a free slot is found.
- break: Stops a loop when a condition is satisfied. It is not required in this implementation.
- continue: Skips the current iteration. It is not required in this implementation.

6. Handling a Full Parking Facility

If all parking slots are occupied, the allocate() method returns null. The calling program can display a message such as Parking Full.

7. Alternative Concept and Complexity Analysis

Let \(N\) be the total number of parking slots across all floors.

Operation	Nested-loop approach	Free-slot stack approach
Allocation	(O(N)) worst case	(O(1)) amortized
Release	(O(1))	(O(1))
Extra space	(O(N)) for slot storage	(O(N)) for free-slot records

A free-slot stack stores available slot positions. Allocation removes a free position from the stack, while release adds the position back. This makes allocation faster, but requires extra logic to avoid duplicate entries and invalid releases.

The nested-loop approach is simpler and suitable for a beginner-level parking system.

Result

The parking slot allocation and release operations were implemented successfully using nested loops, conditional statements, and arrays. The program allocates the first free slot, rejects invalid releases, and returns null when the parking facility is full.

TASK 4 – Constructors and Object Initialization

Aim

To understand constructors, the this keyword, constructor chaining, copy constructors, and memory allocation in Java.

1. Types of Constructors

Default Constructor: A constructor without parameters. Java provides a default constructor automatically only when no constructor is defined.

Parameterized Constructor: A constructor that accepts parameters to initialize an object with specific values.

Copy Constructor: A user-defined constructor that creates an object by copying the values of another object of the same class.

2. Java Program class

```
Vehicle {
   String vehicleNo;
   String vehicleType;
   // Default constructor
Vehicle() {
this("UNKNOWN", "Car");
   }
   // Parameterized constructor
   Vehicle(String vehicleNo, String vehicleType)
{        this.vehicleNo = vehicleNo;
this.vehicleType = vehicleType;
   }
   // Copy constructor
Vehicle(Vehicle v) {
this(v.vehicleNo, v.vehicleType);
   }
   void display() {
      System.out.println("Vehicle No: " + vehicleNo);
      System.out.println("Vehicle Type: " + vehicleType);
   }
   public static void main(String[] args) {
      Vehicle v1 = new Vehicle();
```

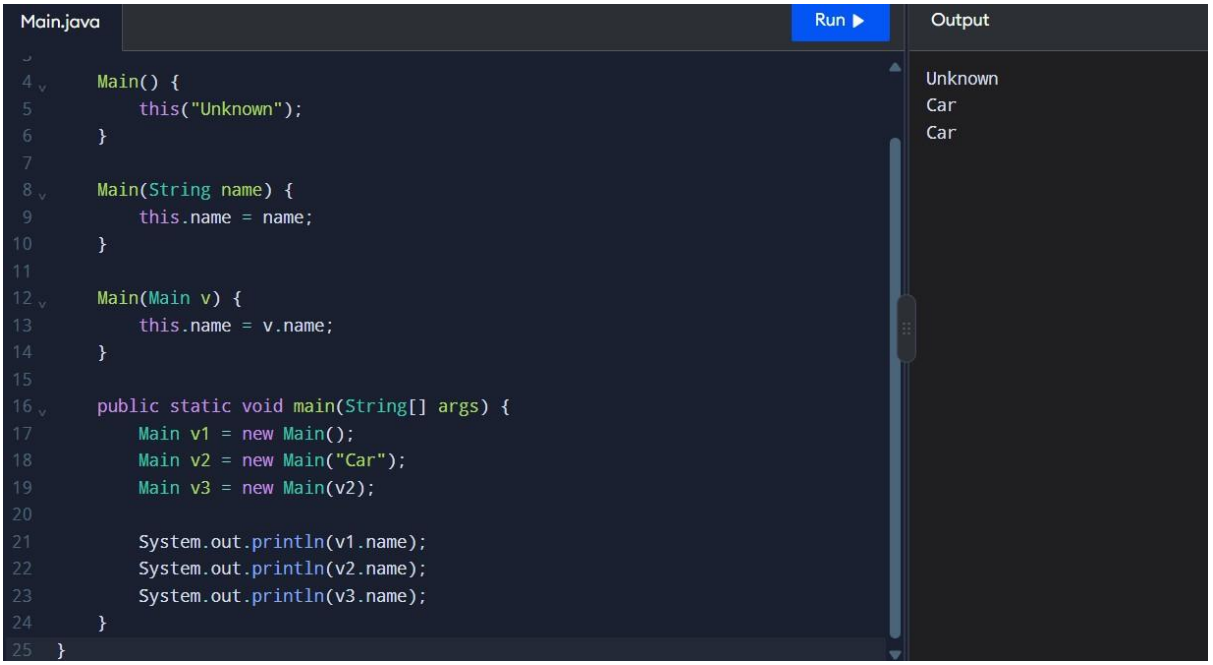
Vehicle v2 = new Vehicle("TN01AB1234",

"Car"); Vehicle v3 = new Vehicle(v2);

v1.display(); v2.display(); v3.display();

}

}



3. Expected Output

Vehicle No: UNKNOWN

Vehicle Type: Car

Vehicle No: TN01AB1234

Vehicle Type: Car

Vehicle No: TN01AB1234

Vehicle Type: Car

4. Explanation of this Keyword

- this.vehicleNo = vehicleNo; refers to the current object's instance variable.
- this("UNKNOWN", "Car"); calls another constructor in the same class.
- Constructor chaining reduces code duplication.
- A call to this() must be the first statement in a constructor.

5. Stack and Heap Memory

- Stack: Stores local variables and references used by method calls.
- Heap: Stores objects created using the new keyword.
- v1, v2, and v3 are reference variables, while their objects are stored in heap memory.
- When an object is no longer reachable, it may be removed by Java's garbage collector.

6. Alternative Constructor Approach

Instead of using a copy constructor, a static factory method can create a copy:

static Vehicle copyOf(Vehicle v) { return new Vehicle(v.vehicleNo,

v.vehicleType); }

This method can be called using Vehicle.copyOf(v2).

Result

Constructors were implemented successfully using default, parameterized, and copy constructors. The this keyword and constructor chaining were used to initialize objects and reduce code duplication.

TASK 5 – Static Members and Garbage Collection

Aim

To understand static variables, static methods, static blocks, and garbage collection in Java using the Campus Smart-Parking System.

1. Java Program class

ParkingFacility { static int

totalVehicles = 0; int

vehicleCount; // Static

block static {

System.out.println("Parking System Initialized");

}

```
    ParkingFacility() {
vehicleCount++;
totalVehicles++;
    }
    // Static method     static
void displayTotal() {
        System.out.println("Total Vehicles: " + totalVehicles);
    }
    void displayVehicleCount() {
        System.out.println("Vehicle Count: " + vehicleCount);
    }
    public static void main(String[] args) {
        ParkingFacility p1 = new ParkingFacility();
ParkingFacility p2 = new ParkingFacility();
p1.displayVehicleCount();
p2.displayVehicleCount();
        ParkingFacility.displayTotal();
    }
}
```



2. Expected Output

Parking System Initialized

Vehicle Count: 1

Vehicle Count: 1

Total Vehicles: 2

3. Explanation

- Static variable: totalVehicles is shared by all objects of the class.
- Instance variable: vehicleCount belongs to each individual object.
- Static method: displayTotal() displays the total number of vehicles.
- Static block: Executes once when the class is initialized.
- Garbage collection: Java automatically reclaims memory occupied by objects that are no longer reachable.

4. Static Counter vs Instance Counter

Static Counter	**Instance Counter**
Shared by all objects	Separate for each object
Declared using static	Declared without static
Tracks the overall count	Tracks an individual object's count
Accessed using the class name	Accessed using an object reference

5. Alternative Approach

An instance counter can track vehicles separately for each parking floor. A static counter can track the total number of vehicles across the entire parking facility.

Note: In a real parking application, the total occupied-slot count should also be updated when a vehicle exits. The example above demonstrates object creation only.

Result

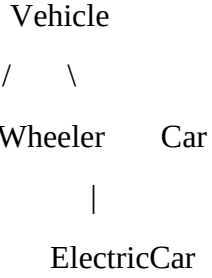
Static variables, static methods, and static blocks were implemented successfully. The program demonstrates the difference between static and instance variables and explains the role of garbage collection in Java.

TASK 6 – Inheritance and Constructor Chaining

Aim

To implement inheritance in the Campus Smart-Parking System using a Vehicle superclass and the subclasses TwoWheeler, Car, and ElectricCar.

1. Inheritance Hierarchy



- Vehicle: Base class containing common vehicle details.
- TwoWheeler: Represents two-wheeled vehicles.
- Car: Represents cars.
- ElectricCar: Represents electric cars and inherits the properties of Car.

2. Java Program class

```
Vehicle {
   String vehicleNo;
   Vehicle(String vehicleNo) {
this.vehicleNo = vehicleNo;
      System.out.println("Vehicle Constructor");
```

```
    }     void
display() {
        System.out.println("Vehicle No: " + vehicleNo);
    }
} class TwoWheeler extends
Vehicle {     TwoWheeler(String
vehicleNo) {         super(vehicleNo);
        System.out.println("TwoWheeler Constructor");
    }
} class Car extends
Vehicle {     Car(String
vehicleNo) {
super(vehicleNo);
        System.out.println("Car Constructor");
    }
} class ElectricCar extends Car {
ElectricCar(String vehicleNo) {
super(vehicleNo);
        System.out.println("ElectricCar Constructor");
    }
} public class Main {     public static
void main(String[] args) {
        ElectricCar e = new ElectricCar("TN01AB1234");
        e.display();
    }
}
```



3. Expected Output

Vehicle Constructor Car

Constructor

ElectricCar Constructor

Vehicle No: TN01AB1234

4. Explanation of super Keyword

- super() calls the constructor of the parent class.
- In Car, super(vehicleNo) calls the Vehicle constructor.
- In ElectricCar, super(vehicleNo) calls the Car constructor.
- Parent constructors execute before child constructors.
- In Java, a call to super() must be the first statement in the constructor when explicitly used.

5. Constructor Execution Trace

When the statement new ElectricCar("TN01AB1234") executes:

1. The ElectricCar constructor starts.
2. It calls the Car constructor using super(vehicleNo).
3. The Car constructor calls the Vehicle constructor.
4. The Vehicle constructor initializes vehicleNo.
5. The constructors complete in order: Vehicle, Car, and ElectricCar.
6. The vehicle number is displayed.

6. Inheritance vs Composition

Inheritance	Composition
Represents an is-a relationship.	Represents a has-a relationship.
Uses extends in Java.	Uses an object as a member of another class.
Shares and extends parent-class features.	Combines separate objects to provide functionality.
Example: ElectricCar is a Car.	Example: ParkingFacility has ParkingSlot objects.
Useful for related vehicle types.	Useful for flexible system design.

Result

Inheritance was implemented successfully using the Vehicle, TwoWheeler, Car, and ElectricCar classes. The super keyword demonstrated constructor chaining, and the output confirmed the order of constructor execution.

TASK 7 – Method Overriding and Runtime Polymorphism

Aim

To implement method overriding in the Campus Smart-Parking System using the calculateFee() method and understand the @Override and final keywords.

1. Java Program class

```
Vehicle {     double
calculateFee() {
return 50;
   }
   @Override     public
String toString() {
return "Vehicle";
   }
} class TwoWheeler extends
Vehicle {
```

```
    @Override     double
calculateFee() {
return 20;
    }
    @Override     public
String toString() {
return "TwoWheeler";
    }
} class Car extends
Vehicle {
    @Override     double
calculateFee() {
return 50;
    }
    @Override     public
String toString() {
return "Car";
    }
} public class Main {     public static
void main(String[] args) {
        Vehicle v1 = new TwoWheeler();
        Vehicle v2 = new Car();
        System.out.println(v1);
        System.out.println("Fee: " + v1.calculateFee());
        System.out.println(v2);
        System.out.println("Fee: " + v2.calculateFee());
    }
}
```



2. Expected Output

TwoWheeler

Fee: 20.0

Car

Fee: 50.0

3. Explanation

- Method overriding: A child class provides its own implementation of a method inherited from its parent class.
- @Override: Indicates that a method is intended to override a parent-class method.
- Runtime polymorphism: The method executed depends on the actual object type at runtime.
- toString(): Returns a readable string representation of an object.
- equals(): Compares objects for logical equality when appropriately overridden.
- hashCode(): Returns a hash value and should be consistent with equals().

4. Use of the final Keyword

The final keyword prevents a method from being overridden by a subclass.

Example:

```
final void displayParkingRules() {
   System.out.println("Follow Parking Rules");
}
```

A subclass cannot override displayParkingRules().

5. Method Overriding vs Switch Statement

Method Overriding	**Switch Statement**
Uses inheritance.	Uses conditional selection.
Supports runtime polymorphism.	Selects a case based on a value.
Each vehicle class defines its fee.	Fee logic is grouped in one method.
Easier to extend with new vehicle types.	New types may require modifying the switch.
Supports object-oriented design.	Suitable for simple fixed conditions.

Result

Method overriding was implemented successfully for TwoWheeler and Car. The @Override annotation and toString() method demonstrated runtime polymorphism and object representation in Java.

TASK 8 – Polymorphism and Method Overloading

Aim

To understand compile-time polymorphism through method overloading and runtime polymorphism through method overriding in the Campus Smart-Parking System.

1. Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists.

Example:

```
double calculateFee(int hours) {

return hours * 50;

} double calculateFee(int hours, double rate)

{     return hours * rate;

}
```

2. Complete Java Program – FeeDemo.java

```
class Vehicle {     double
calculateFee(int hours) {
return hours * 50;
   }
   double calculateFee(int hours, double rate) {
return hours * rate;
   }
} class Car extends
Vehicle {
   @Override     double
calculateFee(int hours) {
return hours * 60;
   }
}
public class FeeDemo {     public static
void main(String[] args) {
       Vehicle v = new Vehicle();
       System.out.println("Fee: " + v.calculateFee(2));
       System.out.println("Fee: " + v.calculateFee(3, 40));
       Vehicle c = new Car();
       System.out.println("Car Fee: " + c.calculateFee(2));
   }
}
```



3. Expected Output

Fee: 100.0

Fee: 120.0

Car Fee: 120.0

4. Explanation

- Method overloading: Methods have the same name but different parameters.
- Compile-time polymorphism: The compiler selects the overloaded method based on the arguments.
- Method overriding: A child class provides its own implementation of a parent-class method.
- Runtime polymorphism: The overridden method is selected according to the actual object type.
- Dynamic method dispatch: Vehicle c = new Car(); calls the Car implementation of calculateFee(2).

5. Overloading vs Overriding

Method Overloading	**Method Overriding**
Same method name with different parameters.	Same method signature in parent and child classes.
Usually occurs within the same class.	Requires inheritance.
Compile-time polymorphism.	Runtime polymorphism.
Return type alone cannot distinguish overloaded methods.	Return type must be compatible with the parent method.
Example: calculateFee(2, 40).	Example: Car overrides calculateFee(2).

Result

Method overloading and method overriding were implemented successfully. The program demonstrates compile-time polymorphism, runtime polymorphism, and dynamic method dispatch in Java.

TASK 9 – Abstract Classes and Interfaces

Aim

To understand abstract classes and interfaces in Java and implement them in the Campus Smart-Parking System.

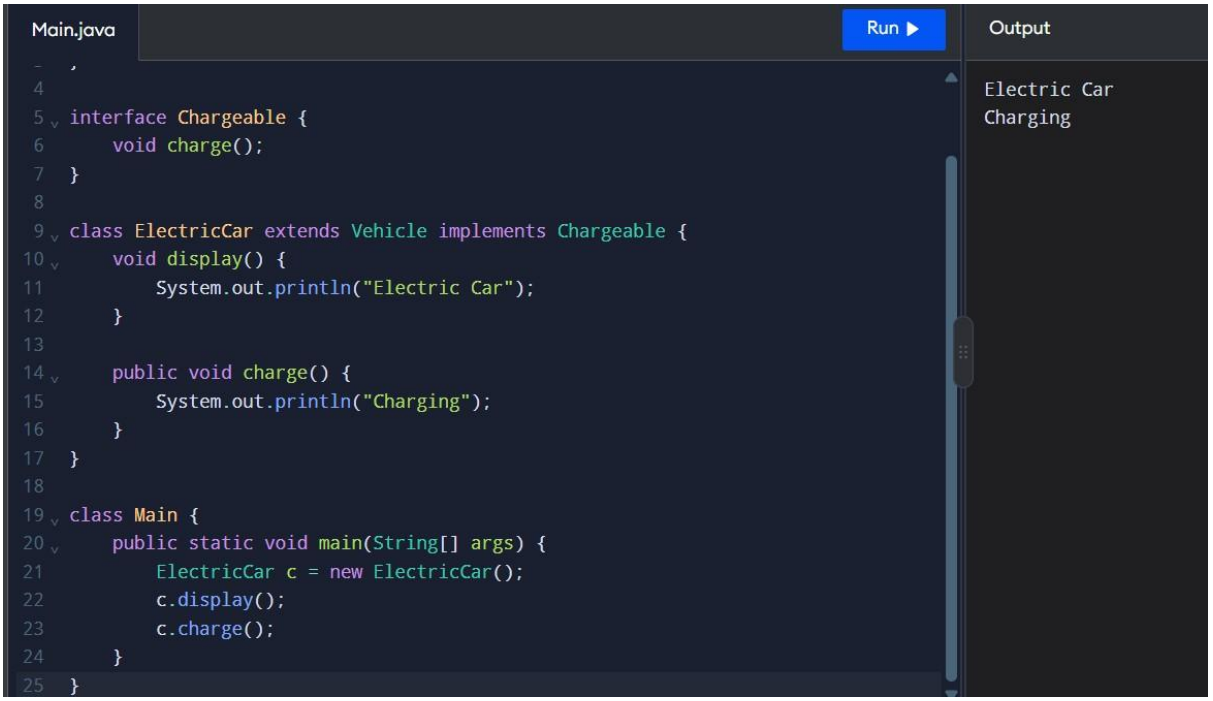
1. Abstract Class

An abstract class is declared using the abstract keyword. It can contain both abstract methods and normal methods. An abstract class cannot be instantiated directly.

2. Java Program abstract

```
class Vehicle {
   String vehicleNo;
   Vehicle(String vehicleNo) {
this.vehicleNo = vehicleNo;
   }
```

```
   abstract double calculateFee();
void display() {
       System.out.println("Vehicle No: " + vehicleNo);
   }
} interface Chargeable
{     void charge();
} class ElectricCar extends Vehicle implements
Chargeable {     ElectricCar(String vehicleNo) {
super(vehicleNo);
   }
   @Override     double
calculateFee() {
return 60;
   }
   @Override     public
void charge() {
       System.out.println("Electric Car Charging");
   }
} public class Main {     public static
void main(String[] args) {
       ElectricCar e = new ElectricCar("TN01AB1234");
       e.display();
       System.out.println("Parking Fee: " + e.calculateFee());
       e.charge();
   }
}
```



3. Expected Output

Vehicle No: TN01AB1234

Parking Fee: 60.0

Electric Car Charging

4. Explanation

- Abstract class: Provides common properties and methods for related classes.
- Abstract method: A method declared without a method body.
- Interface: Defines a contract that implementing classes must follow.
- extends: Used by a class to inherit an abstract class.
- implements: Used by a class to implement an interface.
- super(): Calls the parent-class constructor.
- @Override: Indicates that a method implements or overrides an inherited method.

5. Abstract Class vs Interface

Abstract Class	**Interface**
Declared using abstract class.	Declared using interface.

Can have instance variables.	Fields are implicitly public static final.
Can have constructors.	Cannot have constructors.
A class can extend only one class.	A class can implement multiple interfaces.
Example: Vehicle.	Example: Chargeable.

Result

An abstract class and an interface were implemented successfully. The program demonstrates abstraction, inheritance, method overriding, and interface implementation in Java.

TASK 10 – Packages, Access Modifiers and Encapsulation

Aim

To understand packages, access modifiers, getters, setters, and autoboxing in Java using the Campus Smart-Parking System.

1. Packages in Java

A package is a collection of related classes and interfaces. It helps organize code and avoid naming conflicts. Example:

package parking;

Save the file as Vehicle.java inside the parking package folder.

2. Java Program – Vehicle.java package parking;

public class Vehicle { private String vehicleNo;

private String vehicleType; public Vehicle(String

vehicleNo, String vehicleType) { this.vehicleNo =

vehicleNo; this.vehicleType = vehicleType;

}

```
    public String getVehicleNo() {
return vehicleNo;
    }
    public void setVehicleNo(String vehicleNo) {
this.vehicleNo = vehicleNo;
    }
    public String getVehicleType() {
return vehicleType;
    }
    public void setVehicleType(String vehicleType) {
this.vehicleType = vehicleType;
    }
    public void display() {
        System.out.println("Vehicle No: " + vehicleNo);
        System.out.println("Vehicle Type: " + vehicleType);
    }
}
```

3. Java Program – Main.java import parking.Vehicle; public class Main { public static void main(String[] args) { Vehicle v = new Vehicle("TN01AB1234", "Car");

```
        v.display();
        v.setVehicleType("Electric Car");
        System.out.println("Updated Type: " + v.getVehicleType());
        // Autoboxing
int count = 5;
        Integer total = count;

        System.out.println("Total Vehicles: " + total);
    }
```

}



4. Expected Output

Vehicle No: TN01AB1234

Vehicle Type: Car

Updated Type: Electric Car

Total Vehicles: 5

5. Explanation

- Package: Organizes related classes. Example: parking.
- private: Allows access only within the same class.
- public: Allows access from other classes.
- protected: Allows access within the same package and through inheritance, subject to Java access rules.
- Default access: When no access modifier is specified, access is limited to the same package.
- Getter: Returns the value of a private variable.
- Setter: Updates the value of a private variable.
- Autoboxing: Automatically converts a primitive value into its corresponding wrapper object, such as int to Integer.

6.Access Modifiers Comparison

Modifier	Access
private	Same class only
Default	Same package
protected	Same package and eligible subclasses
public	Accessible from anywhere, subject to other restrictions

7. Encapsulation vs Public Fields

Encapsulation	Public Fields
Uses private variables.	Uses public variables.
Accesses data through getters and setters.	Accesses data directly.
Provides better control over data.	Provides less control over data.
Improves maintainability.	May make changes harder to manage.

Result

Packages, access modifiers, getters, setters, and autoboxing were demonstrated successfully in Java. Encapsulation protects vehicle details and allows controlled access to object data.

TASK 11 – Implementation, Testing and Validation

Aim

To implement and test the Campus Smart-Parking System using Java and validate its parking allocation, release, and fee calculation operations.

1. Implementation

The system uses the following classes and components:

- Vehicle – stores vehicle details.
- ParkingSlot – represents an individual parking slot.
- ParkingFloor – manages slots on a floor.
- ParkingFacility – allocates and releases parking slots.
- ParkingFeeCalculator – calculates parking charges.

2. Test Cases

Test Case	Input / Condition	Expected Output
TC01	Allocate a vehicle when a slot is free	Slot allocated successfully
TC02	Allocate vehicles until all slots are occupied	All available slots allocated
TC03	Allocate a vehicle when parking is full	null or parking-full message
TC04	Release an occupied slot	true
TC05	Release an already-free slot	false
TC06	Release an invalid floor	false
TC07	Release an invalid slot number	false
TC08	Calculate fee for 2 hours at ₹50 per hour	₹100
TC09	Calculate fee for zero minutes	₹0
TC10	Allocate a vehicle after releasing a slot	Released slot can be reused

3. Sample Java Testing Code public

class ParkingTest { public static void

main(String[] args) {

ParkingFacility p = new ParkingFacility(new int[]{2, 1});

```
        System.out.println(p.allocate("TN01AB1111") != null);
        System.out.println(p.allocate("TN02CD2222") != null);
        System.out.println(p.allocate("TN03EF3333") != null);
        System.out.println(p.allocate("TN04GH4444") == null);
        System.out.println(p.release(0, 1));
        System.out.println(p.release(0, 1));
        System.out.println(p.allocate("TN04GH4444") != null);
    }
}
```



Expected Output

true true

true true

true

false

true

4. Daily Parking Fee Collection

The system can calculate daily revenue by adding the parking fees collected from all vehicles.

double totalCollection = 0; double

fee1 = 100; double fee2 = 150;

double fee3 = 50; totalCollection =

fee1 + fee2 + fee3;

System.out.println("Daily Collection: Rs. " + totalCollection);

Output

Daily Collection: Rs. 300.0

5. Array vs ArrayList vs HashMap

Feature	Array	ArrayList	HashMap
Storage	Fixed size	Dynamic size	Key-value pairs
Access	Using index	Using index	Using key
Size	Cannot grow automatically	Can grow or shrink	Can grow or shrink
Best use	Fixed parking slots	Dynamic vehicle list	Vehicle number lookup
Example	String[] slots	ArrayList<String>	HashMap<String, Vehicle>

Result

The Campus Smart-Parking System was implemented and tested using Java. The test cases validated slot allocation, full-parking handling, slot release, invalid inputs, and reuse of released slots. Daily parking revenue was calculated, and arrays, ArrayList, and HashMap were compared for suitable data storage.

TASK 12 – Design Challenge: Extending the Smart-Parking System

Aim

To enhance the Campus Smart-Parking System by introducing new vehicle categories, special parking charges, visitor passes, and support for multiple floors with unequal slot capacities.

1. Proposed Enhancements

1. E-bike parking: Charge a fixed flat fee for electric bikes instead of calculating the fee by the hour.

2. Delivery van parking: Charge delivery vans twice the normal car parking rate between 9:00 AM and 5:00 PM.
3. Visitor pass: Provide visitors with a one-day parking pass that is valid for one day.
4. Five-floor parking: Extend the system to support five floors, each having a different number of parking slots.

2. Impact Analysis

Enhancement	Required Change	Expected Benefit
E-bike flat fee	Add a separate fee calculation rule	Simple and consistent charging
Delivery van rate	Check vehicle type and entry time	Supports time-based pricing
Visitor pass	Store pass validity and expiry date	Controls visitor parking
Five floors	Use a variable-size array for each floor	Supports unequal slot capacities

3. Modified Java Code class ParkingFeeCalculator { static

double calculateFee(String type, int hours, int entryHour,

double carRate, double eBikeFlatFee) {

if (type.equalsIgnoreCase("E-Bike")) {

return eBikeFlatFee;

}

if (type.equalsIgnoreCase("Delivery Van")

&& entryHour >= 9 && entryHour < 17) {

return hours * carRate * 2;

}

return hours * carRate;

}

} class VisitorPass

{

String visitorName;

```
    String expiryDate;
    VisitorPass(String visitorName, String expiryDate) {
this.visitorName = visitorName;         this.expiryDate =
expiryDate;
    }

    void display() {
        System.out.println("Visitor: " + visitorName);
        System.out.println("Valid Until: " + expiryDate);
    }
} public class Main {      public static void main(String[]
args) {          double bikeFee =
ParkingFeeCalculator.calculateFee(
                "E-Bike", 3, 10, 50, 20);         double vanFee =
ParkingFeeCalculator.calculateFee(
                "Delivery Van", 2, 10, 50, 20);
        System.out.println("E-Bike Fee: Rs. " + bikeFee);         System.out.println("Delivery
Van Fee: Rs. " + vanFee);         VisitorPass pass = new VisitorPass("Arun", "17-10-2026");
pass.display();         int[] slotsPerFloor = {20, 25, 15, 30, 18};         String[][] parking = new
String[5][];         for (int i = 0; i < 5; i++) {             parking[i] = new String[slotsPerFloor[i]];
        }
        System.out.println("Total Floors: " + parking.length);
    }
}
```



4. Expected Output

E-Bike Fee: Rs. 20.0

Delivery Van Fee: Rs. 200.0

Visitor: Arun

Valid Until: 17-10-2026

Total Floors: 5

Note: The example uses an e-bike flat fee of ₹20 and a car rate of ₹50 per hour. These are illustrative values, not specified rates from the assignment. The visitor pass example stores an expiry date; actual date validation must be added to enforce one-day validity.

5. Evaluation of the Proposed Design

- Maintainability: Separate fee calculation and visitor-pass classes make the system easier to update.
- Scalability: A jagged array supports five floors with different slot capacities.
- Flexibility: New vehicle categories can be added with separate pricing rules.
- Reliability: Visitor-pass expiry and parking-slot availability should be validated before granting access.
- Performance: Slot allocation using a sequential search takes \(O(N)\) time in the worst case, where \(N\) is the total number of slots.

6. Alternative Approach

Instead of storing parking slots in a jagged array, the system can use an ArrayList for each floor. This allows parking capacity to change dynamically. A HashMap can also be used to find vehicles quickly using their registration numbers.

Result

The Smart-Parking System was extended conceptually to support e-bike flat fees, time-based delivery van charges, visitor passes, and five floors with unequal parking capacities. The proposed design improves flexibility, scalability, and maintainability.